File Upload System (Progress & Retry)

Problem: Build a file upload system that shows per-file progress, retries failed uploads (automatically and manually), and lets the user cancel.

Tech stack: React (Vite, JavaScript) · **Time limit:** 60 minutes

---

PART A: Thought process

Every machine coding round follows the same loop. Say each step out loud so the interviewer can follow and correct you early.

#	Step	What you do
1	Understand requirements	Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions
2	List interactions	Every user action and what the UI should do; this becomes the test checklist
3	Architecture	Components / hooks / services and who talks to whom
4	State model	The minimum state; everything else is derived
5	Contracts	Data shapes and API contracts (mock them when no backend exists)
6	Data flow	Trace one interaction end to end
7	Hard parts	Name the 2-4 tricky problems before you write code
8	Build in layers	Simplest working version first, then add one layer at a time
9	States & edge cases	Loading, empty, error, success, plus problem-specific ones
10	Wrap-up	Trade-offs, what you skipped, what you would add next

Time budget for 60 minutes

Minutes	Activity
0-8	Requirements, questions, assumptions
8-15	Interactions, architecture, state model, contracts
15-45	Build in layers (working app at every step)
45-53	Edge cases, accessibility, polish
53-60	Walk through trade-offs and extensions

Tip: think out loud. The interviewer grades how you reason, not only the final code. Say "I'm keeping the AbortControllers in a ref because they are not UI state" instead of silently typing it.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Select files with a file picker **and** drag-and-drop; multiple files at once.
- Validate files before upload (empty file, max size).
- Show a list with per-file name, size, uploaded bytes, progress bar and status.
- Upload in the background with a **concurrency limit** (only N files at the same time; the rest wait in a queue).
- **Auto-retry** a failed request a few times with exponential backoff.
- **Manual retry** when auto-retry gives up; the retry **resumes** from the last uploaded chunk, it does not restart.
- **Cancel** a queued or running upload; a canceled upload can be resumed.
- Remove an item, "Retry all failed", "Clear completed", overall progress.

Non-functional requirements

Concern	What it means here
Performance	Don't re-render on every byte; update progress per chunk. Concurrency limit protects the browser and server.
Scalability	Chunked upload works for 1 MB and 5 GB alike; queue handles 100 files.
Error handling	Network failure, abort, validation errors are all distinct states with clear messages.
Accessibility	Native `<progress>` with label, `aria-live` status, keyboard-reachable picker and buttons.
Reusability	Upload logic lives in a hook + service; the UI just renders items.
Extensibility	Swap the mock `uploadChunk` for a real XHR / fetch call without touching UI code.

Clarifying questions to ask

1. Is there a real backend, or should I mock the upload API? Does it support chunked / resumable uploads?
2. How many files can upload at the same time? (Concurrency limit.)
3. What are the size / type limits? Should invalid files be rejected silently or with a message?
4. Should retry restart the file or resume from where it stopped?
5. How many automatic retries before we show the error to the user?
6. Should uploads survive a page refresh?

Assumptions we will build with

- No real backend: a **mock** `uploadChunk` with random delay and a 20% failure rate. It has the same signature as a real call.
- Files are uploaded in **256 KB chunks**; the server keeps chunks, so retry resumes from `uploadedChunks`.
- **Max 2 concurrent uploads**, FIFO queue.
- Each chunk is auto-retried **2 times** with backoff 500 ms → 1000 ms; then the file goes to `error`.
- Max file size **100 MB**; empty files rejected; any type allowed.
- No persistence across refresh (mentioned as an extension).

---

2. Identify interactions / use cases

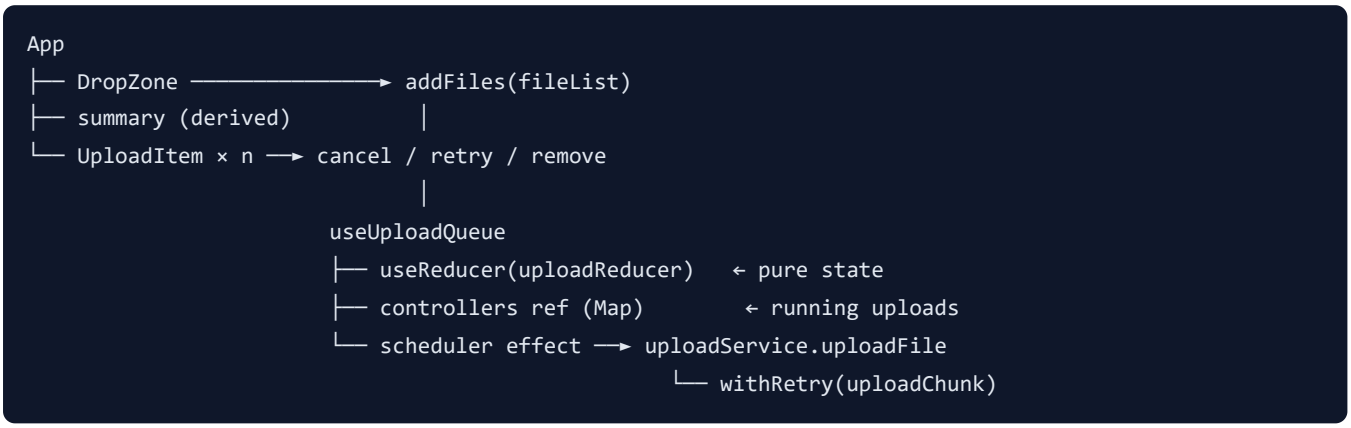
Action / input	Expected behavior / output
Pick or drop 3 valid files	3 items appear; first 2 start `uploading`, third is `queued` (Waiting…)
Drop an empty file / 200 MB file	Not added; red message "empty.txt: file is empty"
Chunk upload succeeds	Progress bar and "x KB / y MB" grow; percent updates
Chunk upload fails once	Retried automatically after a backoff; "auto-retried 1×" shown; no error state
Chunk fails 3 times in a row	Item goes to `error` with message, "Retry/Resume" button appears, slot frees, next queued file starts
Click Retry / Resume	Item goes back to `queued`, then resumes from the last uploaded chunk
Click Cancel on uploading item	Request aborted, item `canceled`, slot frees, next queued starts
Click Cancel on queued item	Item `canceled`, never starts
Click ✕ (remove)	Item removed; if running, its request is aborted first
Click "Retry failed (n)"	All `error` items re-queued
Click "Clear completed"	`success` items removed from list
Select the same file again	Works (input value reset after change)
Component unmounts	All running requests aborted

---

3. Design the architecture

Module	Responsibility
`App`	Layout, derived summary (overall %, counts), rejected-file messages

Module	Responsibility
`DropZone`	File picker + drag-and-drop, hands a `FileList` up
`UploadItem`	Renders one item: progress, status, buttons by status
`useUploadQueue`	Owns state (reducer), scheduler (concurrency), AbortControllers, exposes actions
`uploadReducer`	Pure state transitions + `createUploadItems` validation
`uploadService`	Chunking, `withRetry` backoff, abortable `sleep`, mock `uploadChunk`



Start simple: one component that uploads one file. Split into hook + service only once the queue and retry logic appear, because that is when the responsibilities become clear.

---

4. Design the state / data model

State or field	Purpose
`items[]`	The upload list (reducer state)
`item.id`	Stable key (`crypto.randomUUID()`); file names can repeat
`item.file`	The `File` object, needed to slice chunks
`item.status`	`queued` · `uploading` · `success` · `error` · `canceled`
`item.uploadedBytes`	Drives progress bar
`item.uploadedChunks`	Resume point for retry
`item.totalChunks`	`ceil(size / CHUNK_SIZE)`
`item.autoRetries`	How many automatic retries happened (shown to user)
`item.error`	Last error message
`controllers` (ref)	`Map<id, AbortController>` of running uploads; not UI state
`rejected` (App)	Validation messages from the last add

```
const [items, dispatch] = useReducer(uploadReducer, []);
const controllers = useRef(new Map()); // id -> AbortController

// one item
{
  id: 'b1f…', file, name: 'report.pdf', size: 1572864,
  status: 'uploading', uploadedBytes: 786432, uploadedChunks: 3,
  totalChunks: 6, autoRetries: 1, error: null,
}
```

Senior note

- **Derived, not stored:** percent per item (`uploadedBytes / size`), overall percent, done count, failed count, "running count". Storing them means keeping two sources of truth in sync.
- **Ownership:** the hook owns upload state; `App` only owns `rejected` (pure UI). `UploadItem` is stateless.
- **Why a ref for controllers:** AbortControllers are not serializable, not rendered, and must be readable synchronously by the scheduler. A ref fits; putting them in state would cause extra renders and stale closures.
- **Why `useReducer` :** many transitions (start, progress, retry, cancel…) on one list; a reducer keeps them pure and testable without React.

---

5. Design the data structures and contracts

Example data

```
[
  { id: 'a1', name: 'video.mp4',  size: 52428800, status: 'uploading', uploadedChunks: 40, ... },
  { id: 'a2', name: 'notes.txt',  size: 2048,     status: 'success',   uploadedChunks: 1,  ... },
  { id: 'a3', name: 'notes.txt',  size: 4096,     status: 'queued',    uploadedChunks: 0,  ... },
]
```

Two files named `notes.txt` is why the key is a generated `id` , never the name or the index (index keys break when an item is removed).

API contract (chunked, resumable)

```
POST /uploads                       → { uploadId }          // start session
PUT  /uploads/:uploadId/chunks/:i   body: binary chunk        // idempotent per index
     → 200 { index, received }
     → 5xx / network error  (retry)
POST /uploads/:uploadId/complete    → { url }                // server joins chunks
```

Our mock implements only the chunk call:

```
// Mock server call. Swap for the XHR / fetch version when a real API exists.
export async function uploadChunk(chunk, index, signal) {
```

```
  await sleep(150 + Math.random() * 250, signal);
  if (Math.random() < FAIL_RATE) throw new Error(`Network error on chunk ${index + 1}`);
  return { index, received: chunk.size };
}
```

Real version (same signature, byte-level progress via XHR, because `fetch` has no upload progress event):

```
function uploadChunkXHR(chunk, index, signal, onBytes) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.open('PUT', `/uploads/${uploadId}/chunks/${index}`);
    xhr.upload.onprogress = (e) => onBytes?.(e.loaded);
    xhr.onload = () => (xhr.status < 300 ? resolve(JSON.parse(xhr.response)) : reject(new Error(`HTTP
${xhr.status}`)));
    xhr.onerror = () => reject(new Error('Network error'));
    signal.addEventListener('abort', () => { xhr.abort(); reject(new DOMException('Upload aborted',
'AbortError')); });
    xhr.send(chunk);
  });
}
```

---

6. Trace the data flow

One file, from drop to done, with one failed chunk:

```
User drops report.pdf (1.5 MB)
  ↓
DropZone.onDrop → addFiles(fileList)
  ↓
createUploadItems: validate → item { status: 'queued', uploadedChunks: 0 }
  ↓
dispatch ADD → items updated → re-render
  ↓
scheduler effect: running (controllers.size) = 0 < 2 → runUpload(item)
  ↓
controllers.set(id, AbortController) ; dispatch START → status 'uploading'
  ↓
uploadFile: for chunk 0..5 → withRetry(uploadChunk)
  ↓  chunk 2 throws → onRetry → dispatch AUTO_RETRY → sleep 500 ms → try again → ok
  ↓  each chunk ok → onChunk → dispatch PROGRESS { uploadedChunks, uploadedBytes }
  ↓
all chunks ok → controllers.delete(id) ; dispatch SUCCESS
  ↓
re-render: progress 100%, "Done" ; scheduler effect runs → starts next queued file
```

---

7. Important technical challenges

Challenge 1: Concurrency limit without double-starting

Problem: User drops 10 files. Starting all 10 floods the network. If the scheduler counts "running" from React state, two effect runs before a re-render can both see 1 running and start 2 more each → 3-4 running.

Solution: The `controllers` Map in a ref is the source of truth for "running". `runUpload` adds to it **synchronously** before any `await`, so the next check always sees the correct count. The scheduler runs on every `items` change and fills free slots in FIFO order.

```
useEffect(() => {
  const free = MAX_CONCURRENT - controllers.current.size;
  if (free <= 0) return;
  items.filter((item) => item.status === 'queued').slice(0, free).forEach(runUpload);
}, [items, runUpload]);
```

Challenge 2: Retry with exponential backoff, but never retry a cancel

Problem: A flaky network fails 1 in 5 requests. Showing an error every time is bad UX; retrying instantly in a tight loop hammers the server. And when the user clicks Cancel, the abort error must not be "retried".

Solution: A generic `withRetry` wrapping each chunk: wait `base * 2^attempt`, give up after `retries`, rethrow `AbortError` immediately. The backoff `sleep` is itself abortable, so Cancel during the wait stops instantly.

```
export async function withRetry(fn, { retries, baseDelay, signal, onRetry }) {
  for (let attempt = 0; ; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (err.name === 'AbortError' || attempt >= retries) throw err;
      onRetry?.(attempt + 1);
      await sleep(baseDelay * 2 ** attempt, signal);
    }
  }
}
```

Challenge 3: Resume instead of restart

Problem: A 50 MB file fails at 90% (chunk 180 of 200). Restarting from 0 wastes 45 MB and minutes of the user's time.

Solution: Upload in chunks and store `uploadedChunks` after each success. Manual Retry just re-queues the item; `runUpload` passes `startChunk: item.uploadedChunks`, so only the missing chunks are sent. Retry per chunk also means one bad request costs 256 KB, not the whole file.

```
export async function uploadFile(file, { startChunk = 0, signal, onChunk, onRetry, sendChunk =
uploadChunk }) {
  const totalChunks = Math.ceil(file.size / CHUNK_SIZE);
  for (let i = startChunk; i < totalChunks; i++) {
    const chunk = file.slice(i * CHUNK_SIZE, (i + 1) * CHUNK_SIZE);
    await withRetry(() => sendChunk(chunk, i, signal), { retries: MAX_CHUNK_RETRIES, baseDelay:
BASE_RETRY_DELAY, signal, onRetry });
    onChunk(i + 1, Math.min((i + 1) * CHUNK_SIZE, file.size));
  }
}
```

Challenge 4: Cancel / remove / unmount without leaks or stale updates

Problem: User clicks ✕ on a running upload. The async loop is still awaiting a chunk; when it finishes it dispatches `PROGRESS` / `ERROR` for an item that no longer exists, and the slot is never freed if we wait for the loop.

Solution: One `AbortController` per upload. Cancel/remove abort it **and** free the slot synchronously, then update state. The upload loop sees `AbortError` and returns without dispatching. On unmount, abort everything.

```
const stop = (id) => {
  controllers.current.get(id)?.abort();
  controllers.current.delete(id);
};
// in runUpload's catch:
if (err.name === 'AbortError') return; // cancel/remove already freed the slot and updated state
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite app, `DropZone` with `<input type="file" multiple>`, log files	Files selected
2	`createUploadItems` + reducer `ADD`, render list with name/size	List appears
3	Mock `uploadChunk` + `uploadFile` (no retry), `START/PROGRESS/SUCCESS`	Progress bars move
4	Scheduler effect + `MAX_CONCURRENT` + controllers ref	Only 2 run at once, queue drains
5	`ERROR` state + mock failures	Failures visible
6	`withRetry` + abortable `sleep` + `AUTO_RETRY`	Flaky chunks recover
7	Manual Retry/Resume using `uploadedChunks`	Resume works
8	Cancel / remove with AbortController + unmount cleanup	No leaks
9	Validation messages (empty, too big)	Bad files rejected

#	Step	Working result
10	Drag-and-drop, summary (overall %, Retry failed, Clear completed)	Full feature set
11	Accessibility + styling per status	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial / empty	"No files yet" message
Queued	"Waiting…", 0%, Cancel available
Uploading	Progress bar, bytes, Cancel
Auto-retrying	Still `uploading`, "auto-retried n×" counter
Success	Green, 100%, only remove
Error	Red, message, Retry/Resume, slot freed
Canceled	Dimmed, Resume keeps progress
Empty file	Rejected with message
File > 100 MB	Rejected with message
Same file picked twice in a row	Input value reset so `change` fires again
Duplicate file names	Unique `id`, both upload
Remove while uploading	Abort + free slot, no stale dispatch
Cancel during backoff wait	`sleep` is abortable, stops immediately
Retry many failed at once	Re-queued; scheduler still respects the limit
Unmount mid-upload	All controllers aborted
Last chunk smaller than chunk size	`Math.min((i + 1) * CHUNK_SIZE, size)` keeps bytes exact

10. Final structure

```
_41_fileupload_system_progress_retry/
├── index.html
├── package.json
└── src/
    ├── main.jsx
    ├── App.jsx                    # layout + derived summary
    ├── App.css
    ├── components/
```

```
    │   ├── DropZone.jsx            # picker + drag-and-drop
    │   └── UploadItem.jsx          # one row
    ├── hooks/
    │   └── useUploadQueue.js       # reducer + scheduler + controllers
    ├── state/
    │   └── uploadReducer.js        # pure transitions + validation
    ├── services/
    │   └── uploadService.js        # chunking, retry, mock API
    └── utils/
        └── formatBytes.js
```

---

11. Full working code

src/services/uploadService.js

```
export const CHUNK_SIZE = 256 * 1024; // 256 KB
export const MAX_CHUNK_RETRIES = 2; // auto-retries per chunk before the file fails
export const BASE_RETRY_DELAY = 500; // ms, doubles each attempt
const FAIL_RATE = 0.2; // mock: 20% of chunk requests fail

const abortError = () => new DOMException('Upload aborted', 'AbortError');

// Abortable delay: rejects immediately when the signal fires.
export function sleep(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(abortError());
    const onAbort = () => {
      clearTimeout(timer);
      reject(abortError());
    };
    const timer = setTimeout(() => {
      signal?.removeEventListener('abort', onAbort);
      resolve();
    }, ms);
    signal?.addEventListener('abort', onAbort, { once: true });
  });
}

// Mock server call. Swap for the XHR / fetch version when a real API exists.
export async function uploadChunk(chunk, index, signal) {
  await sleep(150 + Math.random() * 250, signal);
  if (Math.random() < FAIL_RATE) throw new Error(`Network error on chunk ${index + 1}`);
  return { index, received: chunk.size };
}

// Retries fn with exponential backoff. Never retries an abort.
export async function withRetry(fn, { retries, baseDelay, signal, onRetry }) {
  for (let attempt = 0; ; attempt++) {
    try {
```

```
      return await fn();
    } catch (err) {
      if (err.name === 'AbortError' || attempt >= retries) throw err;
      onRetry?.(attempt + 1);
      await sleep(baseDelay * 2 ** attempt, signal);
    }
  }
}

// Uploads chunks from startChunk onwards, so a retry resumes instead of restarting.
export async function uploadFile(file, { startChunk = 0, signal, onChunk, onRetry, sendChunk =
uploadChunk }) {
  const totalChunks = Math.ceil(file.size / CHUNK_SIZE);
  for (let i = startChunk; i < totalChunks; i++) {
    const chunk = file.slice(i * CHUNK_SIZE, (i + 1) * CHUNK_SIZE);
    await withRetry(() => sendChunk(chunk, i, signal), {
      retries: MAX_CHUNK_RETRIES,
      baseDelay: BASE_RETRY_DELAY,
      signal,
      onRetry,
    });
    onChunk(i + 1, Math.min((i + 1) * CHUNK_SIZE, file.size));
  }
}
```

src/state/uploadReducer.js

```
import { CHUNK_SIZE } from '../services/uploadService.js';

export const MAX_FILE_SIZE = 100 * 1024 * 1024; // 100 MB

export function createUploadItems(fileList) {
  const items = [];
  const rejected = [];
  for (const file of fileList) {
    if (file.size === 0) rejected.push(`${file.name}: file is empty`);
    else if (file.size > MAX_FILE_SIZE) rejected.push(`${file.name}: larger than 100 MB`);
    else
      items.push({
        id: crypto.randomUUID(),
        file,
        name: file.name,
        size: file.size,
        status: 'queued', // queued | uploading | success | error | canceled
        uploadedBytes: 0,
        uploadedChunks: 0,
        totalChunks: Math.ceil(file.size / CHUNK_SIZE),
        autoRetries: 0,
        error: null,
      });
  }
```

```
  return { items, rejected };
}

const update = (state, id, patch) => state.map((item) => (item.id === id ? { ...item, ...patch } :
item));

export function uploadReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return [...state, ...action.items];
    case 'START':
      return update(state, action.id, { status: 'uploading', error: null });
    case 'PROGRESS':
      return update(state, action.id, {
        uploadedChunks: action.uploadedChunks,
        uploadedBytes: action.uploadedBytes,
      });
    case 'AUTO_RETRY':
      return state.map((item) =>
        item.id === action.id ? { ...item, autoRetries: item.autoRetries + 1 } : item,
      );
    case 'SUCCESS':
      return update(state, action.id, { status: 'success' });
    case 'ERROR':
      return update(state, action.id, { status: 'error', error: action.error });
    case 'RETRY':
      return update(state, action.id, { status: 'queued', error: null });
    case 'RETRY_ALL_FAILED':
      return state.map((item) => (item.status === 'error' ? { ...item, status: 'queued', error: null }
: item));
    case 'CANCEL':
      return update(state, action.id, { status: 'canceled' });
    case 'REMOVE':
      return state.filter((item) => item.id !== action.id);
    case 'CLEAR_COMPLETED':
      return state.filter((item) => item.status !== 'success');
    default:
      return state;
  }
}
```

src/hooks/useUploadQueue.js

```
import { useCallback, useEffect, useReducer, useRef } from 'react';
import { uploadFile } from '../services/uploadService';
import { createUploadItems, uploadReducer } from '../state/uploadReducer';

export const MAX_CONCURRENT = 2;

export function useUploadQueue() {
  const [items, dispatch] = useReducer(uploadReducer, []);
```

```
  // id -> AbortController for running uploads; also the source of truth for "how many are running".
  const controllers = useRef(new Map());

  const runUpload = useCallback(async (item) => {
    const { id } = item;
    const controller = new AbortController();
    controllers.current.set(id, controller);
    dispatch({ type: 'START', id });
    try {
      await uploadFile(item.file, {
        startChunk: item.uploadedChunks,
        signal: controller.signal,
        onChunk: (uploadedChunks, uploadedBytes) => dispatch({ type: 'PROGRESS', id, uploadedChunks,
uploadedBytes }),
        onRetry: () => dispatch({ type: 'AUTO_RETRY', id }),
      });
      controllers.current.delete(id);
      dispatch({ type: 'SUCCESS', id });
    } catch (err) {
      if (err.name === 'AbortError') return; // cancel/remove already freed the slot and updated state
      controllers.current.delete(id);
      dispatch({ type: 'ERROR', id, error: err.message });
    }
  }, []);

  // Scheduler: fill free slots with queued items, oldest first.
  useEffect(() => {
    const free = MAX_CONCURRENT - controllers.current.size;
    if (free <= 0) return;
    items.filter((item) => item.status === 'queued').slice(0, free).forEach(runUpload);
  }, [items, runUpload]);

  // Abort everything on unmount so no request outlives the component.
  useEffect(() => {
    const running = controllers.current;
    return () => running.forEach((controller) => controller.abort());
  }, []);

  const stop = (id) => {
    controllers.current.get(id)?.abort();
    controllers.current.delete(id);
  };

  const addFiles = useCallback((fileList) => {
    const { items: newItems, rejected } = createUploadItems(fileList);
    if (newItems.length) dispatch({ type: 'ADD', items: newItems });
    return rejected;
  }, []);

  return {
    items,
    addFiles,
    cancel: (id) => {
```

```
      stop(id);
      dispatch({ type: 'CANCEL', id });
    },
    remove: (id) => {
      stop(id);
      dispatch({ type: 'REMOVE', id });
    },
    retry: (id) => dispatch({ type: 'RETRY', id }),
    retryAllFailed: () => dispatch({ type: 'RETRY_ALL_FAILED' }),
    clearCompleted: () => dispatch({ type: 'CLEAR_COMPLETED' }),
  };
}
```

src/utils/formatBytes.js

```
export function formatBytes(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
}
```

src/components/DropZone.jsx

```
import { useState } from 'react';

export default function DropZone({ onFiles }) {
  const [dragging, setDragging] = useState(false);

  const handleDrop = (e) => {
    e.preventDefault();
    setDragging(false);
    onFiles(e.dataTransfer.files);
  };

  return (
    <label
      className={`dropzone ${dragging ? 'dropzone--active' : ''}`}
      onDragOver={(e) => {
        e.preventDefault();
        setDragging(true);
      }}
      onDragLeave={() => setDragging(false)}
      onDrop={handleDrop}
    >
      <input
        type="file"
        multiple
        className="visually-hidden"
        onChange={(e) => {
          onFiles(e.target.files);
```

```
          e.target.value = ''; // allow picking the same file again
        }}
      />
      <strong>Drag files here</strong> or click to browse
      <small>Max 100 MB per file</small>
    </label>
  );
}
```

src/components/UploadItem.jsx

```
import { formatBytes } from '../utils/formatBytes';

const STATUS_LABEL = {
  queued: 'Waiting…',
  uploading: 'Uploading',
  success: 'Done',
  error: 'Failed',
  canceled: 'Canceled',
};

export default function UploadItem({ item, onCancel, onRetry, onRemove }) {
  const percent = Math.round((item.uploadedBytes / item.size) * 100);
  const canCancel = item.status === 'queued' || item.status === 'uploading';
  const canRetry = item.status === 'error' || item.status === 'canceled';

  return (
    <li className={`upload-item upload-item--${item.status}`}>
      <div className="upload-item__row">
        <span className="upload-item__name" title={item.name}>
          {item.name}
        </span>
        <span className="upload-item__meta">
          {formatBytes(item.uploadedBytes)} / {formatBytes(item.size)}
        </span>
      </div>

      <progress value={item.uploadedBytes} max={item.size} aria-label={`${item.name} upload progress`}
/>

      <div className="upload-item__row">
        <span className="upload-item__status" aria-live="polite">
          {STATUS_LABEL[item.status]} · {percent}%
          {item.autoRetries > 0 && ` · auto-retried ${item.autoRetries}×`}
          {item.error && ` · ${item.error}`}
        </span>
        <span className="upload-item__actions">
          {canCancel && <button onClick={() => onCancel(item.id)}>Cancel</button>}
          {canRetry && <button onClick={() => onRetry(item.id)}>{item.uploadedChunks ? 'Resume' :
'Retry'}</button>}
          <button onClick={() => onRemove(item.id)} aria-label={`Remove ${item.name}`}>
```

```
            ×
          </button>
        </span>
      </div>
    </li>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import DropZone from './components/DropZone';
import UploadItem from './components/UploadItem';
import { MAX_CONCURRENT, useUploadQueue } from './hooks/useUploadQueue';
import './App.css';

export default function App() {
  const { items, addFiles, cancel, remove, retry, retryAllFailed, clearCompleted } = useUploadQueue();
  const [rejected, setRejected] = useState([]);

  // Derived, never stored.
  const active = items.filter((i) => i.status !== 'canceled');
  const totalBytes = active.reduce((sum, i) => sum + i.size, 0);
  const uploadedBytes = active.reduce((sum, i) => sum + i.uploadedBytes, 0);
  const overall = totalBytes ? Math.round((uploadedBytes / totalBytes) * 100) : 0;
  const failedCount = items.filter((i) => i.status === 'error').length;
  const doneCount = items.filter((i) => i.status === 'success').length;

  return (
    <main className="uploader">
      <h1>File Upload</h1>
      <DropZone onFiles={(files) => setRejected(addFiles(files))} />

      {rejected.length > 0 && (
        <ul className="rejected" role="alert">
          {rejected.map((msg) => (
            <li key={msg}>{msg}</li>
          ))}
        </ul>
      )}

      {items.length === 0 ? (
        <p className="empty">No files yet. Up to {MAX_CONCURRENT} upload at the same time.</p>
      ) : (
        <>
          <section className="summary">
            <span>
              {doneCount}/{items.length} done · {overall}% overall
            </span>
            <span>
              {failedCount > 0 && <button onClick={retryAllFailed}>Retry failed ({failedCount})
```

```
</button>}
              {doneCount > 0 && <button onClick={clearCompleted}>Clear completed</button>}
            </span>
          </section>
          <ul className="upload-list">
            {items.map((item) => (
              <UploadItem key={item.id} item={item} onCancel={cancel} onRetry={retry} onRemove=
{remove} />
            ))}
          </ul>
        </>
      )}
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f5f6f8; color: #1f2328; }
button { font: inherit; padding: 4px 10px; border: 1px solid #c9ced6; border-radius: 6px; background:
#fff; cursor: pointer; }
button:hover { background: #eef1f5; }

.uploader { max-width: 640px; margin: 40px auto; padding: 0 16px; }
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0);
}

.dropzone { display: flex; flex-direction: column; align-items: center; gap: 4px; padding: 32px;
border: 2px dashed #9aa3af; border-radius: 12px; background: #fff; cursor: pointer; }
.dropzone:focus-within, .dropzone--active { border-color: #2563eb; background: #eff5ff; }
.dropzone small { color: #6b7280; }

.rejected { color: #b91c1c; padding-left: 20px; }
.empty { color: #6b7280; text-align: center; }

.summary { display: flex; justify-content: space-between; align-items: center; margin: 20px 0 8px;
gap: 8px; }
```

```
.summary button { margin-left: 8px; }

.upload-list { list-style: none; padding: 0; margin: 0; display: grid; gap: 8px; }
.upload-item { background: #fff; border: 1px solid #e3e6ea; border-left: 4px solid #9aa3af; border-radius: 8px; padding: 10px 12px; }
.upload-item--uploading { border-left-color: #2563eb; }
.upload-item--success { border-left-color: #16a34a; }
.upload-item--error { border-left-color: #dc2626; }
.upload-item--canceled { opacity: 0.7; }
.upload-item__row { display: flex; justify-content: space-between; align-items: center; gap: 8px; }
.upload-item__name { font-weight: 600; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.upload-item__meta, .upload-item__status { font-size: 13px; color: #4b5563; }
.upload-item--error .upload-item__status { color: #b91c1c; }
.upload-item__actions { display: flex; gap: 6px; flex-shrink: 0; }
.upload-item progress { width: 100%; height: 8px; margin: 6px 0; }
```

---

12. Wrap-up

Edge cases handled

Empty and oversized files, duplicate names, re-selecting the same file, cancel while queued / uploading / backing off, remove while uploading, resume after error or cancel, retry-all respecting the concurrency limit, unmount cleanup, uneven last chunk.

Trade-offs to mention

- **Chunk-level progress, not byte-level.** Small files jump 0 → 100%. With the XHR version, `xhr.upload.onprogress` gives byte-level progress inside a chunk.
- **Sequential chunks per file.** Simpler and resume-friendly; parallel chunks per file are faster on high-latency networks but need out-of-order tracking.
- **Resume assumes the server keeps chunks.** Without a resumable backend, Retry must restart at chunk 0.
- **Per-chunk retry count, not per-file.** A long file on a flaky network can retry many times in total; that is usually what you want.
- **No persistence.** A refresh loses the queue (a `File` can't be stored in localStorage).
- **Mock failure rate is random**, so the demo is not deterministic.

What to add with more time

- Unit tests for the reducer and `withRetry` (pure, no React needed), RTL tests for the UI.
- Byte-level progress + upload speed and ETA.
- Pause / resume button (same mechanism as cancel + resume).
- Persist upload sessions (`uploadId` , `uploadedChunks`) in IndexedDB and resume after refresh (user re-selects the file, matched by name + size + lastModified).
- Retry only on retryable errors (network, 5xx, 429 with `Retry-After`), not on 4xx.
- Jitter on backoff, image thumbnails, file type allow-list, `beforeunload` warning while uploading.

- Virtualize the list for hundreds of files.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why chunked upload instead of one request per file?

One request can't resume: a failure at 90% loses everything. Chunks give a natural resume point, cheaper retries (one 256 KB chunk, not the whole file), and they avoid server body-size limits.

Q2. What if the backend does not support chunks?

Set `CHUNK_SIZE` to the file size (one chunk) and Retry restarts from 0. The rest of the system (queue, retry, cancel) stays the same; only the service changes.

Q3. Why only 2 concurrent uploads?

Browsers allow ~6 connections per host on HTTP/1.1; uploads compete with page requests and saturate upload bandwidth anyway, so more parallel uploads don't finish faster. It's a constant, easy to tune.

State design

Q4. Why keep AbortControllers in a ref and not in state?

They are not rendered, not serializable, and the scheduler needs the current count synchronously. State updates are async, so counting running uploads from state can double-start; the ref is updated immediately.

Q5. Why `useReducer` instead of several `useState`s?

All transitions act on one list and depend on the previous item. A reducer makes each transition a pure function I can unit test in Node, and `dispatch` is stable, so callbacks don't need dependencies.

Q6. Why not store `percent` or `isUploading`?

They are derivable from `uploadedBytes / size` and `status`. Stored copies drift out of sync; derived values can't.

Challenge: concurrency

Q7. Your scheduler effect runs on every progress update. Isn't that wasteful?

It's a filter over the list plus a size check; with a few hundred items that's microseconds. If it showed up in a profile, I'd move scheduling into the action handlers (start next on success / error / cancel) instead of an effect.

Q8. What happens in React StrictMode, which runs effects twice?

StrictMode double-runs effects only on mount, when the list is empty. Later runs check `controllers.size`, which `runUpload` updates synchronously, so a second run would see no free slots. The reducer is pure, so double-invoking it is safe.

Challenge: retry

Q9. Why exponential backoff and not a fixed delay?

If the server is overloaded, fixed fast retries from many clients make it worse. Doubling the delay gives it room to recover. In production I'd add jitter (`delay * (0.5 + Math.random())`) so clients don't retry in sync.

Q10. Should every error be retried?

No. Network errors, 5xx and 429 are retryable; 400 / 401 / 413 won't succeed on retry. I'd tag errors with `retryable` in the service and have `withRetry` rethrow non-retryable errors immediately, the same way it handles `AbortError`.

Q11. What if the retry succeeds on the server but the response is lost?

The client retries a chunk the server already has. That's why the chunk endpoint is `PUT .../chunks/:index`: idempotent, writing the same index twice is harmless.

Challenge: resume and cancel

Q12. How do you resume after a page refresh?

`File` objects can't be persisted, so the user must re-select the file. Store `{ uploadId, name, size, lastModified, uploadedChunks }` in IndexedDB; on re-select match the fingerprint, or ask the server `GET /uploads/:id` for received chunks, and start from there.

Q13. What happens if the user clicks Cancel during the backoff wait?

`sleep` listens to the same `AbortSignal`, clears its timer and rejects with `AbortError`; `withRetry` rethrows aborts without retrying, so the loop exits immediately.

Q14. Why free the slot in `cancel` instead of in a `finally` block?

The aborted promise settles asynchronously. If the slot were freed in `finally`, the scheduler effect from the `CANCEL` render could run first, see the slot still taken, and not start the next file until another state change.

Implementation steps

Q15. How would you show real byte-level progress?

`fetch` has no upload progress events, so use `XMLHttpRequest` and `xhr.upload.onprogress`. Report `chunkStart + e.loaded` and throttle the dispatch (e.g. with `requestAnimationFrame`) so we don't re-render on every event.

Q16. How do you test this?

Reducer and `withRetry` are pure: plain unit tests. `uploadFile` takes `sendChunk` as a parameter, so tests pass a fake that fails on demand. For the UI, React Testing Library with a mocked service and fake timers to step through backoff.

Performance / scale

Q17. What if the user drops 1,000 files?

The queue keeps only 2 running, so the network is fine. The list renders 1,000 rows and every progress update re-maps the array: virtualize the list (`react-window`), wrap `UploadItem` in `React.memo` (unchanged items keep the same object reference), and maybe collapse finished items.

Q18. What if one file is 5 GB?

Chunking already avoids loading it into memory: `file.slice` is lazy. 20,000 chunks of 256 KB is a lot of requests, so bump the chunk size to 5-10 MB and send 2-3 chunks of the same file in parallel.

Q19. What if the API is slow (10 s per chunk)?

Progress still moves per chunk but feels stuck; switch to byte-level progress, add a per-request timeout (abort and retry), and show speed/ETA so the user knows it's alive.

Trade-offs / "why not X?"

Q20. Why not a library like Uppy or tus-js-client?

In production I'd seriously consider them (tus is a standard resumable protocol). In an interview the point is to show the queue, retry and abort mechanics myself; the service layer is where a library would plug in.

Q21. Why not Redux or Context?

Only one screen uses this state. A custom hook keeps it local and portable; if another part of the app needed the upload list (e.g. a global upload tray), I'd lift the hook into a Context provider without changing its API.

Q22. Why not upload with a Web Worker or Service Worker?

A Service Worker with Background Fetch could continue uploads after the tab closes, but support is limited. The main thread isn't the bottleneck here: network I/O is async, and slicing a file is cheap.